

GIBRAN IGLESIAS

Resumen de evaluación técnica

EJERCICIO
atyt_assistant_challenge

FECHA
30 de julio de 2026

MÉTODO
Código, ejecución en vivo y forense de git

☒ AGENDAR SESIÓN TÉCNICA — LA DEFENSA DECIDE

~71 / 100 sobre lo auditable (el 80%). El 20% restante es la defensa, y es eliminatoria.

En una frase: Implementó, delegó, seleccionó, solucionó y debugó hasta que arrancara. Una vez que arrancó, se cercioró de cumplir con el examen — no de terminar lo que hizo.

Lo que hizo bien

Los problemas de fondo del ejercicio están resueltos, y se verificó ejecutándolo:

- Detectó la inyección de prompt del enunciado y la defendió estructuralmente (filtro por tenant en SQL, no en el prompt).
- --workers 1 con la razón escrita en cuatro archivos → evitó el error estrella del ejercicio.
- PRAGMA de SQLite condicionales al motor → no se ató a SQLite ni al motor destino.
- Datos de ejemplo reales: 40 tenants con volúmenes desiguales, ~4.000 pedidos, PDF de 400 páginas.
- Streaming SSE con eventos tipados, citas verificables construidas en servidor, escalado a ticket.

Las tres pruebas en vivo, en verde: arranca sin API key · aguanta peticiones simultáneas · se puede cancelar sin fugas.

⚠ Pero casi todo eso es la diana que el propio examen pintó

Son literalmente las respuestas ☒ que el manual de evaluación esperaba. Cumplir el objetivo no es destacar. Por encima de la rúbrica solo hay tres cosas de peso:

1. Sembró la inyección en el corpus de forma permanente, con el argumento escrito de que borrarla escondería el problema. El examen pedía detectarla, no conservarla.
2. Backpressure real en el limitador: cola acotada, timeout, rechazo rápido y métrica expuesta. El examen solo pedía el semáforo.
3. Eliminó el tool-calling por completo — el modelo nunca elige qué ejecutar. Es una respuesta distinta a la del manual, más fuerte en seguridad, con coste real en rigidez.

Ninguna de las tres prueba autoría propia. Una conversación guiada con las preguntas correctas llega al mismo sitio.

Lo que está mal

Severidad	Problema	Detalle
ALTO	El asistente no dice nada	Todas las respuestas salen vacías por una colisión de delimitadores con su propio system prompt. No abrió su propia aplicación.
ALTO	Sin trazabilidad	No persiste conversaciones. El modelo de datos para hacerlo ya existía en el repo, sin conectar.
ALTO	Sin conjunto de evaluación	No puede saber si un cambio mejora o empeora la recuperación. No está declarado en NO_HICE.md: punto ciego, no priorización.
MEDIO	E/S bloqueante	Toda consulta a base de datos corre sobre el único event loop. Medido: $\times 1.7$ de latencia a 20 concurrentes.
BAJO	Frontend sin revisar	Un render de React por token, más scroll suave ~ 40 veces por segundo. Valores por defecto sin tocar.

Y cuatro afirmaciones de su documentación que el código no sostiene — todas describiendo el sistema como más completo de lo que es.

El patrón, que es lo que importa

Mira qué bugs corrigió y cuáles no:

✓ Corregidos (impiden ARRANCAR)	✗ No corregidos (se ven MIRANDO LA SALIDA)
• bash ausente • pipefail • bit de ejecución • healthcheck incompatible • errata en el nombre	• respuestas vacías • sobre-citación • re-render por token • modelos sin conectar

La línea no es backend / frontend. Es arrancar / mirar.

Depuró con rigor hasta que el sistema levantó. No siguió hasta comprobar que respondía bien.

Y optimizó contra la rúbrica, no contra el producto. La prueba más limpia: entregó exactamente los tres documentos que pedía el enunciado y borró ARQUITECTURA.md —284 líneas con diagrama y flujo, su mejor trabajo— porque no estaba en la lista. Ninguna casilla del examen decía "comprueba que el asistente diga algo".

Lo que este análisis NO puede resolver

Los artefactos no distinguen entre quien decide y quien selecciona bien entre opciones que le ofrecieron. Un seleccionador competente y un autor producen el mismo repositorio.

Sus documentos (DECISIONES.md, NO_HICE.md, AI_LOG.md) no valen como evidencia: se generan en un prompt a partir del código terminado.

Por eso la decisión es la defensa. No falta información técnica — está toda, y es favorable. Falta saber si el criterio es suyo, y si reconoce lo que no verificó.

Cómo conducir la defensa

Preguntar por decisiones documentadas no sirve: se memoriza releendo el repo. Hay que sondear alrededor.

1. El vecindario, no el punto. “Tu WAL lo pone Django. Arranco el contenedor de la API primero, con base recién creada y Django sin haber corrido nunca. ¿Qué pasa?”
2. Números, no adjetivos. “¿Cuánto tarda esa consulta? ¿Cuánta RAM habrían costado los embeddings que descartaste?” Quien midió tiene cifras.
3. Que ataque su propia decisión. “Convénceme de que --workers 1 es mala idea.” Quien eligió de un menú conoce los pros; le cuesta el contraargumento fuerte.
4. Que decida algo nuevo en vivo. “Ahora el asistente puede emitir reembolsos.” El criterio se ve en qué necesita saber antes de elegir.
5. Abrirle su propia aplicación y preguntarle dónde está el contenido de la respuesta.

Guion completo con la respuesta correcta escrita y hoja de calificación: §8.1 y §8.2 de gibran-analisis.md.

Regla de decisión

Si en la defensa...	Entonces
Sostiene, extiende y ataca sus decisiones, y reconoce sin rodeos lo que no verificó	☑ Contratar. Da igual de quién fuera la idea: puede sostenerla.
Sostiene solo lo escrito en su repo · se queda en adjetivos · no sabe atacar su elección	⚠ Seleccionó, no decidió. Solo con supervisión de entregables.
No explica el mecanismo de sus decisiones clave · minimiza el bug	✗ No contratar.

No cambia en ningún escenario: el hueco de verificación y el punto ciego en evaluación y trazabilidad de sistemas de IA. Eso ya no es hipótesis, y necesita mentoría explícita.

Resumen de gibran-analisis.md. Todas las afirmaciones sobre el código están ancladas a archivo y línea y son reproducibles. Capturas de las vistas: captura-01...13.png.